

Department of Computer and Software Engineering
SE: Machine Learning

Course Instructor: Dr. Ahmed Raza	Date: 26-February-2026
Day: Thursday	Semester: 6th

Lab 3: Hard Margin SVM — Geometry of Maximum Margin

Lab Title	CLO	BT	PLO	Weightage
Hard Margin SVM: Geometry of Maximum Margin				

Name	Roll Number	Report /15	Viva /5	Total /20
Muhammad Abdul Qadir	BSSE23105			

Checked on: _____

Signature: _____

1 Learning Outcomes

After completing this lab, the student will be able to:

- Explain the maximum margin principle.
- Compute geometric margin width.
- Verify hard-margin constraints numerically.
- Identify support vectors and interpret their role.
- Analyze geometric sensitivity to scaling and point removal.

2 Equipment Required

- PC with Python 3.x
- NumPy
- Matplotlib
- scikit-learn

3 Theory and Background

Hard Margin SVM solves:

$$\min_{w,b} \frac{1}{2} \|w\|^2$$

Subject to:

$$y_i(w^T x_i + b) \geq 1$$

Geometric margin width:

$$\frac{2}{\|w\|}$$

Support vectors satisfy:

$$y_i(w^T x_i + b) = 1$$

Only these points determine the separating boundary.

4 Part A — Training a Hard Margin Classifier

Task A1: Generate Separable Dataset (2 Marks)

Implementation Box

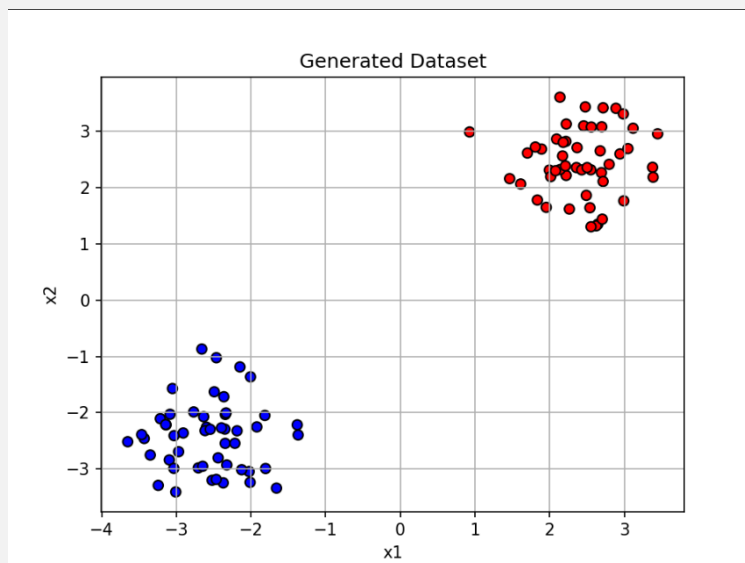
Generate a 2D linearly separable dataset using a fixed random seed.
Plot the dataset with labels.

Code for generating dataset

```
def generate_dataset(n=100, seed=42):  
    """  
    Generate a 2D linearly separable dataset.  
  
    Requirements:  
    - Use fixed random seed  
    - Labels must be {-1, +1}  
    - Return X (n,2), y (n,)  
    """  
  
    np.random.seed(seed)  
  
    # Splitting in half positive and half negative  
    n_positive = n // 2  
  
    n_negative = n - n_positive  
  
    # Classes centers  
    center_positive = np.array([2.5, 2.5])  
  
    center_negative = np.array([-2.5, -2.5])  
  
    # Standard deviation  
    spread = 0.6  
  
    positive_points = center_positive + spread * np.random.randn(n_positive, 2)  
  
    negative_points = center_negative + spread * np.random.randn(n_negative, 2)  
  
    X = np.vstack([positive_points, negative_points])  
  
    y = np.hstack([np.ones(n_positive, dtype=int), -1 * np.ones(n_negative, dtype=int)])  
  
    return X, y
```

Code for generating dataset

```
if __name__ == "__main__":  
  
    # Step 1  
    X, y = generate_dataset()  
  
    # Raw Visualization of data  
    plt.figure()  
    plt.scatter(X[:, 0], X[:, 1], c=y, cmap='bwr', edgecolors='k')  
    plt.title("Generated Dataset")  
    plt.xlabel("x1")  
    plt.ylabel("x2")  
    plt.grid(True)  
    plt.show()
```

Result/Plot:**Task A2: Train Linear SVM (2 Marks)****Implementation Box**

Train a linear SVM using sklearn:

`SVC(kernel='linear')`

This model will compute the maximum-margin separating hyperplane.

Code:

```
def train_linear_svm(X, y):  
    """  
    Train a linear SVM using sklearn.  
  
    Instructions:  
    - Use kernel='linear'  
    - Fit the model  
    - Return trained model  
  
    DO NOT modify dataset.  
    """  
  
    # Using a very large C for identifying hard-margin SVM for separable data  
    model_svm = SVC(kernel='linear', C=1e6)  
  
    model_svm.fit(X, y)  
  
    return model_svm
```

Task A3: Extract Parameters (2 Marks)**Analysis Box**

Extract w and b .
Print w .

Code

```
5 def extract_parameters(model):
6     """
7     Extract:
8         w (weight vector)
9         b (bias term)
10
11     Hint:
12         model.coef_
13         model.intercept_
14
15     Return:
16         w, b
17     """
18
19     weight = model.coef_[0].copy()
20
21     bias_term = float(model.intercept_[0])
22
23     norm_w = np.linalg.norm(weight)
24
25     print(" ||w|| ---> ", norm_w)
26
27     return weight, bias_term
```

Output (Print):

```
||w|| ---> 0.4060147990602201
```

5 Part B — Verifying Hard Constraints

Task B1: Constraint Verification (3 Marks)

Analysis Box

Compute $y_i(w^T x_i + b)$.

Report minimum value.

Verify all values ≥ 1 .

Code:

```
def verify_constraints(margins):  
    """  
    Print:  
    - Minimum margin value  
    - Whether all margins >= 1  
  
    DO NOT modify margins.  
    """  
  
    min_value = margins.min()  
  
    # Normal strict check  
    strict_check = np.all(margins >= 1.0)  
  
    # Small tolerance is needed because floating point calculations  
    # sometimes give values like 0.99999999 instead of exactly 1  
    tol_check = np.all(margins >= 1.0 - 1e-8)  
  
    print("Minimum margin value:", min_value)  
  
    print("Without tolerance ->", strict_check)  
  
    print("With tolerance ->", tol_check)  
  
    print("\n")
```

Output:

```
Minimum margin value: 0.999999992625839  
Without tolerance -> False  
With tolerance -> True
```

Task B2: Margin Visualization (2 Marks)**Implementation Box**

Compute geometric margin width.
Plot margin lines $w^T x + b = \pm 1$.

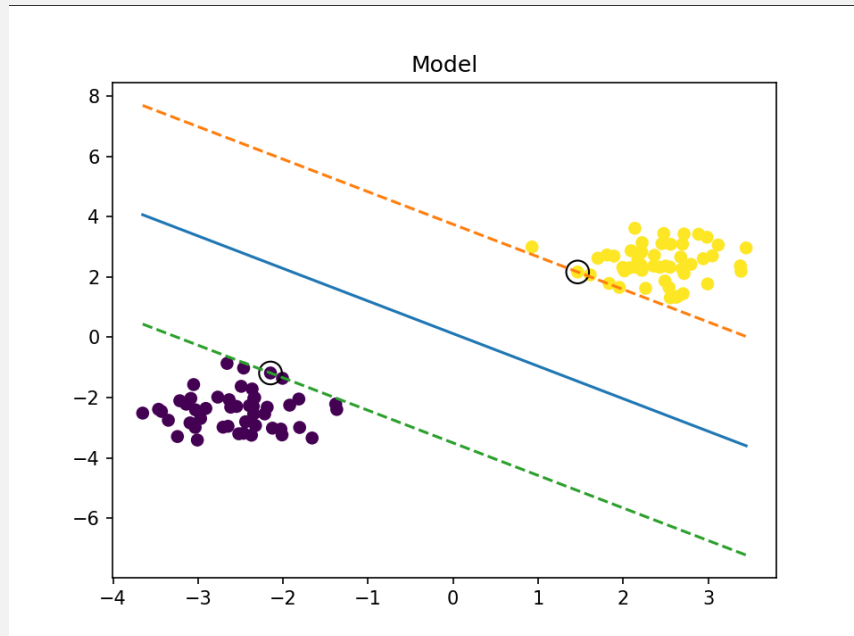
Code:

```
def compute_margin_width(w):  
    """  
    Compute geometric margin width:  
        2 / ||w||  
  
    Return scalar.  
    """  
  
    norm_w = np.linalg.norm(w)  
  
    if norm_w == 0:  
        return float('inf')  
    return 2.0 / norm_w
```

Plot Code:

```
# Margin lines  
y_margin1 = -(w[0]*x_vals + b - 1) / w[1]  
y_margin2 = -(w[0]*x_vals + b + 1) / w[1]  
  
plt.plot(x_vals, y_margin1, linestyle='--')  
plt.plot(x_vals, y_margin2, linestyle='--')
```

Plot:



6 Part C — Support Vectors

Task C1: Identify Support Vectors (3 Marks)

Analysis Box

Print support vector indices and coordinates.
Verify numerically that they satisfy margin equality.

Code:

```
def analyze_support_vectors(model, X, y, w, b):
    """
    Tasks:

    1. Print indices of support vectors.
    2. Print number of support vectors.
    3. Verify numerically that support vectors
        approximately satisfy:
         $y_i (w^T x_i + b) \approx 1$ 

    """

    sv_indices = model.support_.copy()
    sv_points = model.support_vectors_.copy()
    num_sv = sv_points.shape[0]

    print("Support vector indices:", sv_indices)
    print("\n")

    print("Number of support vectors:", num_sv)
    print("\n")

    print("Support vector coordinates:\n", sv_points)
    print("\n")
    margins_for_sv = []

    for idx in sv_indices:
        x_i = X[idx]
        y_i = y[idx]
        margin_value = y_i*(w.dot(x_i)+b)
        margins_for_sv.append(margin_value)

    margins_for_sv = np.array(margins_for_sv)
    print("Margins for support vectors (should be ≈ 1):", margins_for_sv)
    print("All support margins approx equal to 1 (tol=1e-6):", np.allclose(margins_for_sv, np.ones_like(margins_for_sv), atol=1e-6))
```

Output and verification

Support vector indices: [62 7]

Number of support vectors: 2

Support vector coordinates:
[[-2.14788574 -1.18572662]
[1.4650493 2.16262748]]

Margins for support vectors (should be ≈ 1): [0.99999999 1.00000001]
All support margins approx equal to 1 (tol=1e-6): True

Task C2: Visualization (2 Marks)

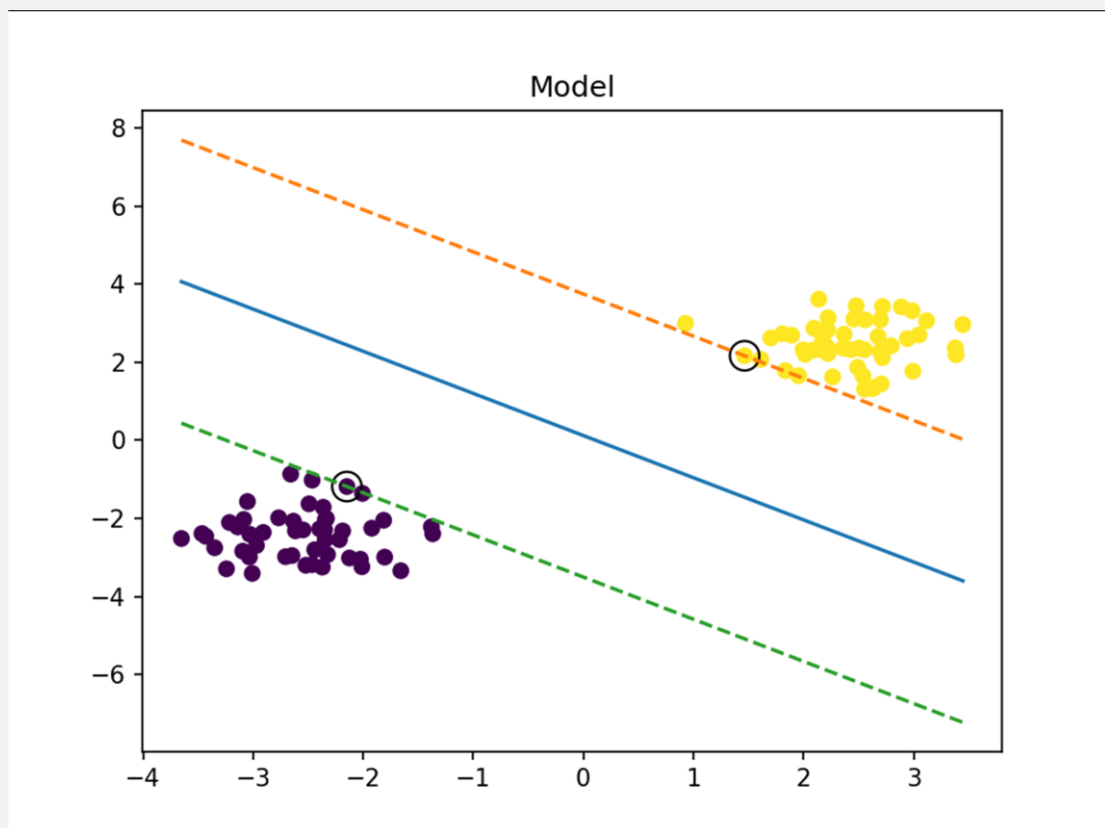
Implementation Box

Highlight support vectors on the plot.

Code:

```
# Highlight support vectors  
plt.scatter(model.support_vectors_[0], model.support_vectors_[1], s=150, facecolors='none', edgecolors='black')
```

Plot:



7 Part D — Removing a Support Vector (2 Marks)

Experiment Box

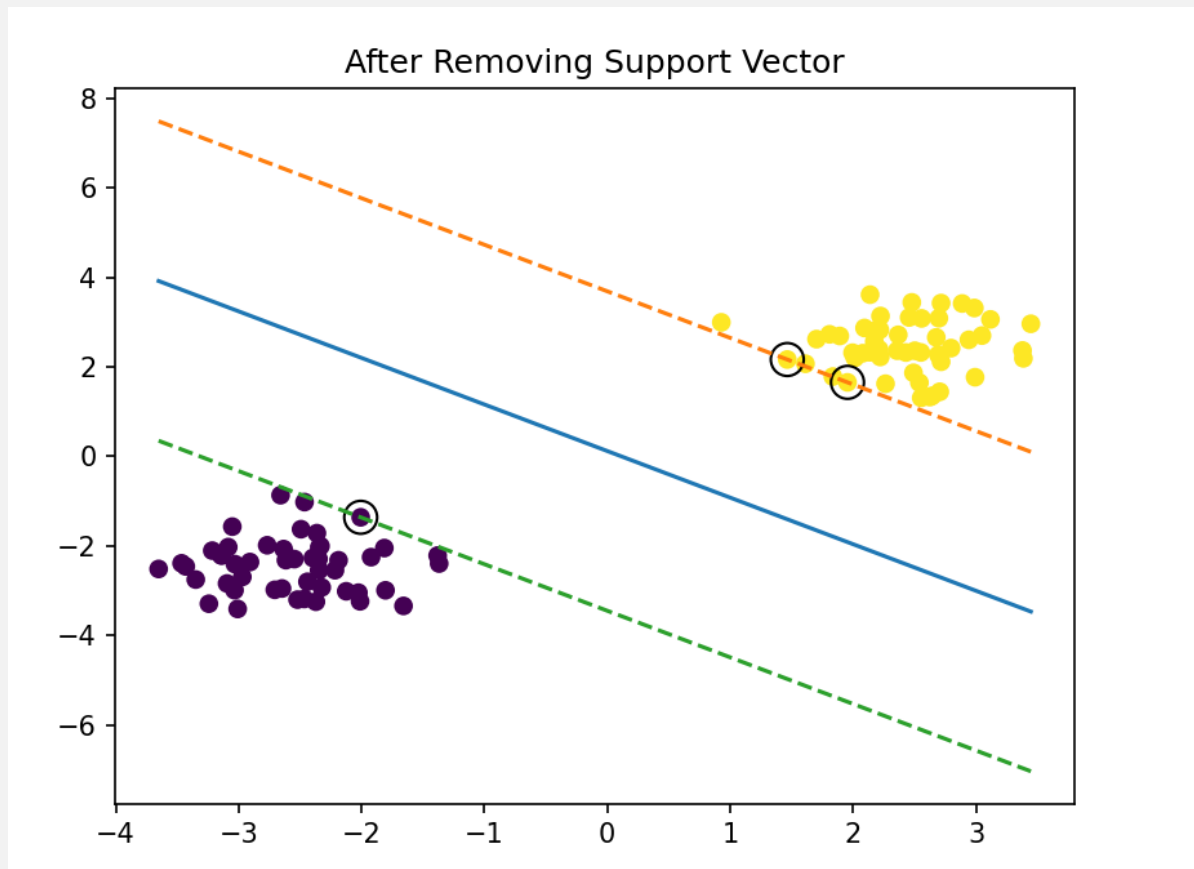
Remove one support vector.
Retrain model.
Plot new boundary and compare.

Code:

```
def remove_one_support_vector(model, X, y):  
    """  
    Remove the first support vector from dataset.  
  
    Steps:  
    1. Identify index using model.support_  
    2. Remove from X and y  
    3. Return new X, y  
  
    DO NOT retrain inside this function.  
    """  
    if model.support_.size == 0:  
        return X.copy(), y.copy()  
  
    idx_to_remove = int(model.support_[0])  
  
    keep = np.ones(X.shape[0], dtype=bool)  
  
    keep[idx_to_remove] = False  
  
    X_reduced = X[keep].copy()  
    y_reduced = y[keep].copy()  
  
    print(f"Removed support vector at original index: {idx_to_remove}")  
  
    return X_reduced, y_reduced
```

Retraining

```
# Step 7  
X_new, y_new = remove_one_support_vector(model, X, y)  
model_new = train_linear_svm(X_new, y_new)  
plot_model(X_new, y_new, model_new, title="After Removing Support Vector")
```

Plot:**Comparison:**

Observation: After removing the support vector (index 62), the strict limits on the margin were reduced.

Result: The solid blue line (decision boundary) moved slightly, and the gap between the dashed lines (the margin) became wider.

Reasoning: Support vectors act like pillars holding up the margin. When one was removed, the model could expand the margin further until it touched the next closest points. This shows that the SVM model depends heavily on these specific points to find the best boundary.

8 Part E — Scaling Experiment (2 Marks)

Experiment Box

Multiply all features by 10.
Retrain model.
Compute new margin.
Explain change.

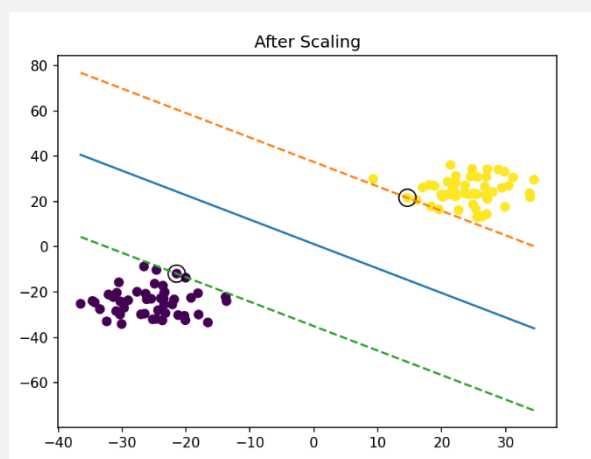
Code:

```
def scale_dataset(X, factor):  
    """  
    Multiply features by scaling factor.  
  
    Return scaled X.  
    """  
  
    return (X*factor).copy()
```

Multiplying and Retraining:

```
# Step 8  
X_scaled = scale_dataset(X, factor=10)  
model_scaled = train_linear_svm(X_scaled, y)  
plot_model(X_scaled, y, model_scaled, title="After Scaling")
```

Plot:



Compute new margin.

```
# Step 8
X_scaled = scale_dataset(X, factor=10)
model_scaled = train_linear_svm(X_scaled, y)

w_scaled, b_scaled = extract_parameters(model_scaled)
new_margin = compute_margin_width(w_scaled)
print("New geometric margin width:", new_margin)

plot_model(X_scaled, y, model_scaled, title="After Scaling")
```

Output

```
||w||---> 0.04060147891644802
New geometric margin width: 49.25928939967215
```

Change Explanation:

- **Scaling the Data:** When we multiplied the dataset by 10, the data points moved 10 times further apart, but their pattern stayed exactly the same.
- **Geometric Change:** The decision boundary (the line separating the groups) did not change its angle. It still separates the points in the exact same way.
- **Mathematical Reason:** The margin width is calculated using the formula $2 / ||w||$. Because the input numbers are now 10 times larger, the weight vector (w) must become 10 times smaller to keep the equation balanced.
- **Result:** Since w is 10 times smaller, the margin width becomes 10 times bigger, The margin got wider simply because we zoomed out the data scale. The actual classification logic remained identical.

9 Reflection (2 Marks)

Reflection Box

1. Why do only support vectors determine the solution?
2. Why does scaling change margin but not classification?

Answer#1

Support vectors are the specific data points that lie exactly on the edges of the margin boundaries. They can be thought of as pillars holding up the margin, because the model uses these points to define the widest possible gap between the two groups. Any data point that is not a support vector lies safely far from the boundary. If these “safe” points are moved, the boundary does not need to change because they are not limiting the margin width. Since only the support vectors restrict how wide the margin can be, the final solution depends entirely on them.

Answer#2

When we multiply the dataset by 10, the points move further apart, but their relative arrangement stays exactly the same. Because the overall pattern remains identical, the decision boundary does not need to change its angle or direction and it still separates the classes correctly. The margin width is calculated as $2 / \|w\|$. When the input values become 10 times larger, the weight vector w must become 10 times smaller to keep the equation balanced. As w becomes smaller, the margin width becomes larger. Therefore, the margin grows when the data is scaled, but the classification logic remains unchanged.